

PORTADA

Técnicas de Programación

Unidad 7 — Desarrollo Web: HTML: Formularios, Tablas y Semántica

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



TRANSICIÓN

Retomando la sesión anterior

La sesión pasada vimos qué es el desarrollo web, cómo conversan cliente y servidor con HTTP, y armamos la estructura mínima de un documento HTML: `<html>` , `<head>` , `<body>` . Con eso ya sabes escribir texto, listas, enlaces e imágenes — y ya interiorizaste que el atributo `alt` en las imágenes **no es opcional**, es accesibilidad básica.

Todo eso construye páginas que **se leen**. Hoy damos el siguiente paso: páginas con las que el usuario **interactúa** (formularios), datos organizados en filas y columnas (tablas), y una forma de escribir HTML que no solo se ve bien, sino que **comunica su propia estructura** — a un lector de pantalla, a un buscador y a la próxima persona que lea tu código.

CONCEPTO

El elemento `<form>`

Un **formulario** es el mecanismo con el que una página HTML recolecta datos del usuario y se los envía a un servidor para que los procese (guardarlos en una base de datos, iniciar sesión, buscar algo). Todo formulario empieza envolviendo sus campos en una etiqueta `<form>` .

Atributo	Qué define
<code>action</code>	La URL del servidor que va a recibir los datos cuando se envía el formulario
<code>method</code>	Cómo se envían esos datos: <code>GET</code> o <code>POST</code>

A alto nivel: `GET` envía los datos visibles en la URL (útil para búsquedas, nada sensible) y `POST` los envía "por dentro" de la petición, sin exponerlos en la URL (el método normal para registros, login o cualquier dato sensible). El detalle completo de cómo el servidor recibe y procesa esto queda para cursos posteriores de backend — aquí nos enfocamos en cómo se construye el formulario del lado del cliente.

CONCEPTO

Campos de un formulario: <input>

Dentro de <form> , el elemento más usado es <input> . Su atributo type cambia por completo cómo se comporta y qué controles muestra el navegador:

type	Para qué sirve
text	Texto libre de una sola línea (nombre, dirección)
email	Correo electrónico — el navegador valida el formato básico
password	Texto oculto con puntos o asteriscos, para contraseñas
number	Solo valores numéricos, con flechas para incrementar/decrementar
checkbox	Casilla de sí/no, se pueden marcar varias a la vez
radio	Botón de opción única entre varias (mismo name agrupa las opciones)

CONCEPTO

<label> : la etiqueta que no es decorativa

Cada campo necesita una **etiqueta** que diga qué se debe escribir ahí. La forma correcta es `<label for="id-del-campo">` , asociada al `id` del `<input>` correspondiente — no basta con poner el texto al lado sin conectarlo.

```
<label for="correo">Correo electrónico</label>
<input type="email" id="correo" name="correo">
```

Esta asociación importa por dos razones concretas: hace **clik en el texto** y activa el campo (útil para checkboxes y radios, más fáciles de "acertar" con el mouse), y un **lector de pantalla** anuncia el texto de la etiqueta cuando el usuario llega al campo con el teclado — sin eso, la persona escucha "campo de texto" sin saber para qué es.

CONCEPTO

Más allá de <input> : selección y texto largo

Elemento	Para qué sirve
<select> / <option>	Lista desplegable de opciones; cada <option> es una alternativa seleccionable
<textarea>	Caja de texto de varias líneas (comentarios, mensajes largos)
<button>	Botón de acción; type="submit" envía el formulario

```
<label for="pais">País</label>
<select id="pais" name="pais">
  <option value="co">Colombia</option>
  <option value="mx">México</option>
</select>

<label for="comentario">Comentario</label>
<textarea id="comentario" name="comentario" rows="4"></textarea>

<button type="submit">Enviar</button>
```

CONCEPTO

Ejemplo completo: formulario de contacto

```
<form action="/enviar-mensaje" method="POST">
  <label for="nombre">Nombre</label>
  <input type="text" id="nombre" name="nombre" required>

  <label for="correo">Correo</label>
  <input type="email" id="correo" name="correo" required>

  <label for="edad">Edad</label>
  <input type="number" id="edad" name="edad" min="1" max="120">

  <label>
    <input type="checkbox" name="suscribir"> Suscribirme al boletín
  </label>

  <label for="mensaje">Mensaje</label>
  <textarea id="mensaje" name="mensaje" rows="4" required></textarea>

  <button type="submit">Enviar</button>
</form>
```

CONCEPTO

Validación básica sin JavaScript

El ejemplo anterior ya usa **validación nativa de HTML**: el propio navegador revisa ciertas condiciones antes de dejar enviar el formulario, sin necesitar una sola línea de código adicional.

Atributo	Qué valida
required	El campo no puede quedar vacío
type="email"	El texto debe tener forma de correo (algo@algo.algo)
min / max	Límites numéricos aceptados en type="number"

Esta validación es un primer filtro útil, pero **no es suficiente por sí sola**: siempre debe reforzarse del lado del servidor, porque un usuario puede desactivar JavaScript o enviar la petición sin pasar por el formulario. Cómo agregar validaciones más complejas con JavaScript es tema de sesiones posteriores de esta misma unidad.

CONCEPTO

Tablas: `<table>` para datos tabulares

Una **tabla** organiza información en filas y columnas — el elemento correcto cuando los datos realmente tienen esa forma (una lista de estudiantes con sus notas, un horario de clases, un catálogo de productos).

Etiqueta	Qué representa
<code><table></code>	Contenedor de toda la tabla
<code><thead></code>	Sección del encabezado (nombres de columnas)
<code><tbody></code>	Sección con los datos propiamente dichos
<code><tr></code>	Una fila (<i>table row</i>)
<code><th></code>	Celda de encabezado (<i>table header</i>), va dentro de <code><thead></code>
<code><td></code>	Celda de datos (<i>table data</i>), va dentro de <code><tbody></code>

CONCEPTO

Ejemplo: tabla de calificaciones

```
<table>
  <thead>
    <tr>
      <th>Estudiante</th>
      <th>Nota</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Ana Torres</td>
      <td>4.5</td>
    </tr>
    <tr>
      <td>Luis Pérez</td>
      <td>3.8</td>
    </tr>
  </tbody>
</table>
```

Este bloque mismo se está mostrando en una tabla generada con esta sintaxis — así se ve el resultado en pantalla: filas y columnas claras, con el encabezado visualmente diferenciado de los datos.

CONTRA EJEMPLO

Usar `<table>` para maquetar toda la página

Antes de que existiera CSS moderno, era común usar tablas **para organizar el layout completo** de una página (menú a la izquierda en una celda, contenido en otra, pie de página en una fila final) — no porque los datos fueran tabulares, sino como truco visual.

```
<table>
  <tr>
    <td>Menú</td>
    <td>Contenido principal</td>
  </tr>
  <tr>
    <td colspan="2">Pie de página</td>
  </tr>
</table>
```

Esta práctica rompe la **accesibilidad** (un lector de pantalla anuncia "tabla de datos" sobre algo que no tiene datos, confundiendo al usuario) y el **diseño responsivo** (las tablas no se reorganizan bien en pantallas pequeñas, algo que verás en la sesión de CSS más adelante). Hoy el layout de una página se maqueta con etiquetas semánticas y CSS — nunca con tablas.

CONCEPTO

Etiquetas semánticas: HTML que se explica solo

Las etiquetas **semánticas** describen el *significado* de una sección de la página, no solo su apariencia — a diferencia de `<div>` , que es un contenedor genérico sin ningún significado propio.

Etiqueta	Qué representa
<code><header></code>	Cabecera de la página o de una sección (logo, título, navegación)
<code><nav></code>	Bloque de enlaces de navegación principal
<code><main></code>	Contenido principal de la página (uno solo por página)
<code><section></code>	Agrupación temática de contenido, normalmente con su propio título
<code><article></code>	Contenido independiente y autocontenido (un post, una noticia)
<code><aside></code>	Contenido relacionado pero secundario (barra lateral, nota al margen)
<code><footer></code>	Pie de página o de una sección (créditos, enlaces legales)

CONCEPTO

Por qué importa: accesibilidad, SEO y mantenibilidad

Un **lector de pantalla** es un software que convierte el contenido de una página en voz o braille, usado por personas con discapacidad visual para navegar la web. Cuando el HTML usa etiquetas semánticas, el lector de pantalla puede anunciar "navegación principal" o "contenido principal" y permitir saltar directo entre secciones — algo imposible si todo es `<div>` sin distinción.

- **Accesibilidad:** lectores de pantalla entienden la estructura y permiten navegación rápida entre secciones.
- **SEO:** los buscadores usan estas etiquetas para entender qué es el contenido principal frente al menú o el pie de página, y lo priorizan mejor en resultados.
- **Mantenibilidad:** un archivo con `<header>` , `<main>` , `<footer>` se lee y navega mucho más rápido que un archivo con veinte `<div>` genéricos sin nombre.

CONCEPTO

Accesibilidad básica más allá de alt

Ya conoces `alt` en imágenes. Otras tres prácticas básicas completan una página accesible desde el HTML:

- **aria-label** : un atributo que describe un elemento para lectores de pantalla cuando no hay texto visible que lo explique (por ejemplo, un botón que solo tiene un ícono).
- **Contraste**: el texto debe distinguirse claramente de su fondo — un color de texto muy parecido al fondo es ilegible para personas con baja visión (esto se ajusta con CSS, próxima sesión).
- **Jerarquía de encabezados**: usar `<h1>` , `<h2>` , `<h3>` ... en orden, sin saltar niveles (nunca pasar de `<h1>` directo a `<h4>`) — un lector de pantalla usa esa jerarquía para armar un índice navegable de la página.

```
<button aria-label="Cerrar ventana">  
  <svg>...</svg>  
</button>
```

INTERACCIÓN

Reestructuren este HTML con etiquetas semánticas

En parejas: este fragmento funciona, pero usa `<div>` para todo. Reescribanlo usando las etiquetas semánticas correctas de esta sesión.

```
<div class="cabecera">
  <div class="logo">Mi Blog</div>
  <div class="menu">
    <a href="/">Inicio</a>
    <a href="/blog">Blog</a>
  </div>
</div>
<div class="contenido">
  <div class="post">
    <h2>Mi primer artículo</h2>
    <p>Texto del artículo...</p>
  </div>
</div>
<div class="pie">© 2026 Mi Blog</div>
```

8 minutos · pista: ¿cuál `<div>` es en realidad un `<nav>` ? ¿cuál es `<article>` y cuál `<main>` ?

SÍNTESIS

Lo que hay que llevarse de hoy

1. Un formulario (`<form>`) recolecta datos con `<input>` , `<select>` , `<textarea>` y `<button>` ; cada campo necesita un `<label for>` correctamente asociado.
2. `required` , `type="email"` y `min / max` dan validación básica sin escribir JavaScript.
3. Las tablas (`<table>` , `<thead>` , `<tbody>` , `<tr>` , `<th>` , `<td>`) son para datos tabulares — nunca para maquetar el layout completo de una página.
4. Las etiquetas semánticas (`<header>` , `<nav>` , `<main>` , `<section>` , `<article>` , `<aside>` , `<footer>`) reemplazan al `<div>` genérico y mejoran accesibilidad, SEO y mantenibilidad.
5. Accesibilidad va más allá de `alt : aria-label` , buen contraste y una jerarquía de encabezados sin saltos.



CIERRE

Antes de la próxima sesión

- Practicar un formulario propio con al menos tres tipos de `<input>` distintos y validación nativa (`required` , `type="email"`).
- Reestructurar una página propia (o el ejercicio de hoy) reemplazando todos los `<div>` genéricos por etiquetas semánticas.

Próxima sesión: CSS3 — cómo se ve la página que acabamos de estructurar. Selectores y el modelo de caja para darle estilo real a formularios, tablas y secciones semánticas.

